Gradient Descent: Lecture Notes

Optimisation for machine learning, from first principles

Mohamad Hassan Ibrahim - Artificial Intelligence notes

1. The problem

Training a model means finding parameters that make a loss function small. For a parameter vector w and a loss L(w), we look for the w that minimises L. For most models there is no closed-form solution, so we search iteratively.

2. The update rule

The gradient of L points in the direction of steepest increase of the loss. Gradient descent takes a small step in the opposite direction:

```
w(t+1) = w(t) - eta * grad L(w(t))
```

Here eta > 0 is the learning rate. Repeating this update moves w downhill until the gradient is close to zero.

3. A one-dimensional example

Let L(w) = (w - 3)^2, so the derivative is L'(w) = 2(w - 3) and the minimum is at w = 3. Starting from w = 0 with eta = 0.1:

```
t   w(t)     L'(w(t))   w(t+1)
0   0.0000   -6.0000    0.6000
1   0.6000   -4.8000    1.0800
2   1.0800   -3.8400    1.4640
3   1.4640   -3.0720    1.7712
```

Each step closes 20% of the remaining distance to the minimum, because w(t) - 3 = 0.8^t * (0 - 3). After 20 steps w is about 2.965.

4. Choosing the learning rate

For this loss, every update multiplies the error w - 3 by (1 - 2 * eta). The iteration therefore converges when |1 - 2 * eta| < 1, that is when 0 < eta < 1. With eta = 0.5 it reaches the minimum in a single step. With eta = 1 the error flips sign without shrinking, so w alternates between 0 and 6. With eta > 1 the error grows and the method diverges.

- Too small: progress is slow and training is expensive.
- Too large: the loss oscillates or diverges.
- In practice: try values on a logarithmic scale such as 0.1, 0.01 and 0.001, and watch the training loss.

5. Stochastic and mini-batch variants

When the loss is an average over n training examples, computing the full gradient costs O(n) per step. Stochastic gradient descent (SGD) estimates the gradient from a single example, and mini-batch gradient descent from a small batch, typically between 32 and 512 examples. The estimate is noisy, but each step is far cheaper, and the noise can help the search move past flat regions and shallow local minima.

6. Momentum

Momentum keeps a running velocity v that accumulates past gradients:

```
v(t+1) = beta * v(t) + grad L(w(t))
w(t+1) = w(t) - eta * v(t+1)
```

A typical value is beta = 0.9. Momentum speeds up progress along directions where the gradient is consistent and damps oscillations across narrow valleys. Adaptive methods such as Adam extend the same idea with a separate step size for each parameter.

Summary

- Gradient descent repeatedly steps against the gradient.
- The learning rate controls stability as much as speed.
- Mini-batches trade gradient accuracy for many more updates per second.
- Momentum and adaptive methods are refinements of the same core rule.